Universidad de los Andes
Facultad de Ingeniería

Departamento de Ingeniería de Sistemas y Computación

ISIS - 1222 – Introducción a la Programación Honores

ABET Engineering Accreditation Commission

16 de octubre.

Quiz 2 – N3 – E2

Por favor lea cuidadosamente las siguientes instrucciones:

- Este quiz es **individual** y su duración máxima es de **20 min**.
- Está prohibida cualquier interacción con sus compañeros durante el examen.
- No se permite el uso de dispositivos electrónicos (celulares, relojes inteligentes, agendas electrónicas, computadores, etc.) ni de materiales adicionales como apuntes, libros o impresos.
- **Debe** escribir su respuesta de manera clara y legible, aplicando la sintaxis y los conceptos de Python estudiados en el curso. Se provee una cuadrícula para facilitar la indentación de su código; úsela de manera que quede claro en qué cuadro inicia cada instrucción. **Las respuestas ilegibles o inadecuadamente indentadas pueden ser calificadas como incorrectas.** Puede usar lápiz**, pero únicamente se aceptarán reclamos sobre respuestas escritas en esfero.**
- Ante un intento de copia, el encargado podrá **retirarle el enunciado** y asignar una nota de **0.0** o iniciar un **proceso disciplinario**. Por favor, mantenga la mirada exclusivamente en su enunciado.

Compromiso de integridad:
Al proporcionar sus datos a continuación, usted confirma su compromiso de completar este examen de acuerdo con las normas de integridad académica de la Universidad de los Andes, incluyendo acatar el Reglamento General de Estudiantes de Pregrado y todas las disposiciones relacionadas con evitar el *Fraude académico*[1], establecidas en el Reglamento Disciplinario para Estudiantes de Pregrado.

Nombre: _____

Firma: _____

[1] En el artículo 4 del Reglamento Disciplinario para Estudiantes de Pregrado, se define el *Fraude académico* como "el comportamiento del estudiante que infringe las reglas de la Universidad o las reglas establecidas por el evaluador, en desarrollo de una actividad académica".

Las dos preguntas de este quiz piden implementar la **misma validación** de un password, que se recibe como un string: la Pregunta 1 con un ciclo `for` y la Pregunta 2 con un ciclo `while`. Ambas funciones retornan `True` si el password es válido y `False` si no lo es.

Un password es válido si cumple todas las siguientes condiciones:

1. Tiene como mínimo **8 caracteres**, es decir, 8 o más. Todos los caracteres cuentan, incluidos los espacios.
2. Contiene al menos **uno** de estos **símbolos**: `#`, `$`, `%`, `&` o `!`. Se permiten varias apariciones del mismo símbolo o combinaciones de ellos. Otros símbolos, como `@` o `*`, cuentan para la longitud, pero no para cumplir esta condición.
3. Contiene como mínimo **un dígito** (del `0` al `9`).

Ejemplos (aplican a las dos funciones):

Password	Retorno	Razón
`""`	`False`	Está vacío, así que tiene menos de 8 caracteres.
`"ab#1"`	`False`	Tiene un símbolo y un dígito, pero solo 4 caracteres.
`"abcdefg1"`	`False`	No tiene ninguno de los símbolos permitidos.
`"abcdefg#"`	`False`	No tiene ningún dígito.
`"clave@2026"`	`False`	El símbolo `@` no es uno de los símbolos permitidos.
`"abcdef#1"`	`True`	Tiene exactamente 8 caracteres, un símbolo y un dígito.
`"1234567!"`	`True`	Cumple las tres condiciones; no se exige que tenga letras.
`"Mi Clave & 9"`	`True`	Los espacios también cuentan: tiene 12 caracteres.

Requerimientos para las dos preguntas:

1. **DEBE** revisar cada uno de los caracteres del password, usando exclusivamente el ciclo solicitado y no se permite ninguna otra forma de recorrido.
2. En la Pregunta 1, **DEBE** usar únicamente un ciclo `for`.
3. En la Pregunta 2, **DEBE** usar únicamente un ciclo `while`.
4. **DEBE** usar un único `return` al final de cada función.

IMPORTANTE:

- Incumplir alguno de los anteriores requerimientos resultará en **0 pts** en la pregunta correspondiente.
- Puede usar `len()` y el método `str.isdigit()`. También puede usar el operador de pertenencia (o membresía) `in` para verificar si un carácter está en un string.
- En cada hoja de respuesta ya se proveen la signatura, el *docstring*, una aserción y las constantes que **debe** usar: `LONGITUD_MINIMA = 8` y `SIMBOLOS = "#$%&!"`.
- Debe implementar **únicamente el resto del cuerpo de cada función** en la cuadrícula ubicada a continuación de las constantes dadas.
- No debe crear *doctests* ni aserciones adicionales. No se necesita importar nada.

Pregunta 1 (50 pts). Implemente la función `validar_password_for` usando un ciclo **`for`**.

Rúbrica de calificación de la Pregunta 1:

Descripción	Cumple	Incumple
(15 pts) Recorre con el ciclo `for` los caracteres del password sin omitir ninguno y sin salirse de sus límites.	15	0
(10 pts) Registra si el carácter actual es uno de los símbolos permitidos y conserva ese hallazgo hasta el final, sin reiniciarlo en las iteraciones siguientes.	10	0
(10 pts) Registra si el carácter actual es un dígito y conserva ese hallazgo hasta el final, sin reiniciarlo en las iteraciones siguientes.	10	0
(5 pts) Usa las dos constantes dadas (`LONGITUD_MINIMA` y `SIMBOLOS`).	5	-2.5 por c/constante no usada
(10 pts) Retorna `True` si se cumplen las tres condiciones establecidas a la vez (verificando la longitud mínima de forma inclusiva: 8 caracteres o más) y `False` en caso contrario.	10	0
Total		**_____ / 50**

Pregunta 2 (50 pts). Implemente la función `validar_password_while` usando un ciclo **`while`**.

Rúbrica de calificación de la Pregunta 2:

Descripción	Cumple	Incumple
(5 pts) Antes del ciclo `while`, declara e inicializa la variable de control.	5	0
(10 pts) El ciclo `while` revisa uno a uno los caracteres del password dado, sin omitir ninguno y sin salirse de sus límites.	10	0
(10 pts) Registra si el carácter actual es uno de los símbolos permitidos y conserva ese hallazgo hasta el final, sin reiniciarlo en las iteraciones siguientes.	10	0
(10 pts) Registra si el carácter actual es un dígito y conserva ese hallazgo hasta el final, sin reiniciarlo en las iteraciones siguientes.	10	0
(5 pts) Actualiza la variable de control, de modo que el ciclo siempre termina.	5	0
(5 pts) Usa las dos constantes dadas (`LONGITUD_MINIMA` y `SIMBOLOS`).	5	-2.5 por c/constante no usada
(5 pts) Retorna `True` si se cumplen las tres condiciones establecidas a la vez (verificando la longitud mínima de forma inclusiva: 8 caracteres o más) y `False` en caso contrario.	5	0
Total		**_____ / 50**

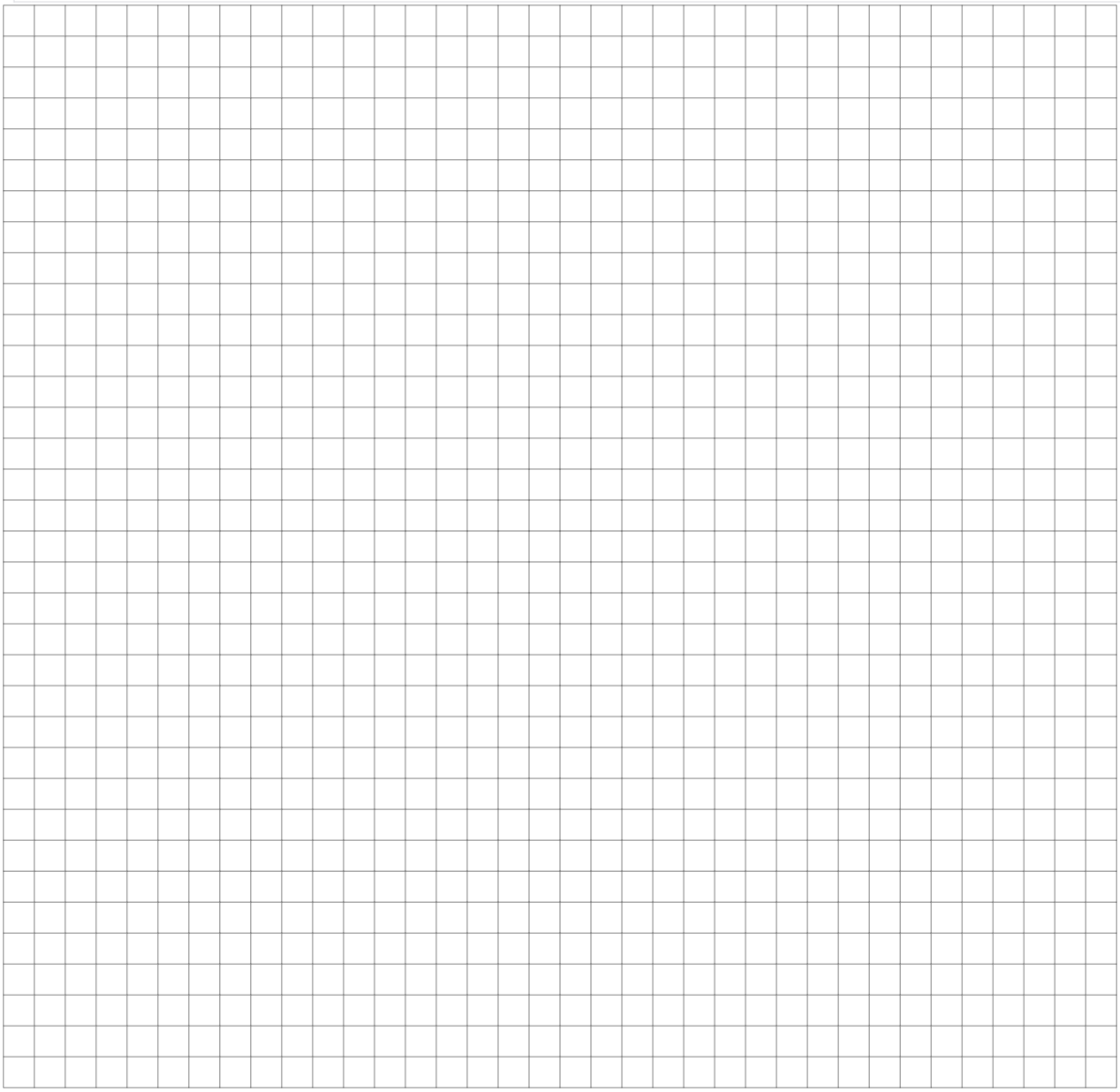
Hoja de respuesta de la Pregunta 1 (ciclo `for`)

```python
def validar_password_for(password: str) -> bool:
    """
    Valida un password usando un ciclo for. El password es válido si cumple TODAS las condiciones:
        1. Tiene como mínimo 8 caracteres.
        2. Contiene como mínimo uno de los siguientes símbolos: #, $, %, &, !
        3. Contiene como mínimo un dígito.

    Args:
        password (str): Password que se desea validar.

    Returns:
        bool: True si el password cumple las tres condiciones, False en caso contrario.
    """
    assert isinstance(password, str), "Se esperaba un password del tipo str, pero se recibió: " +
         str(type(password))

    LONGITUD_MINIMA = 8
    SIMBOLOS = "#$%&!"
```



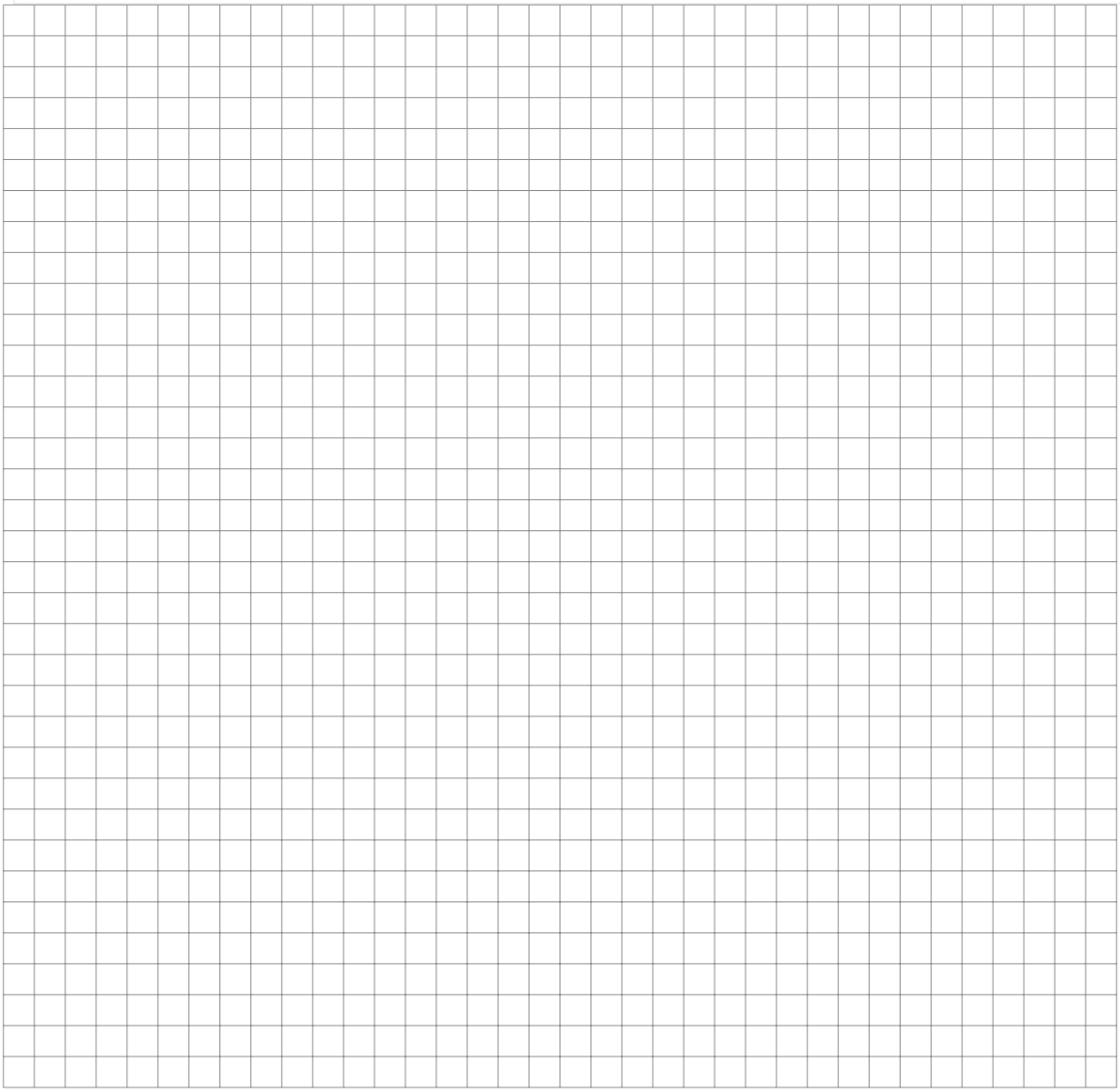
Hoja de respuesta de la Pregunta 2 (ciclo `while`)

```python
def validar_password_while(password: str) -> bool:
    """
    Valida un password usando un ciclo while. El password es válido si cumple TODAS las condiciones:
        1. Tiene como mínimo 8 caracteres.
        2. Contiene como mínimo uno de los siguientes símbolos: #, $, %, &, !
        3. Contiene como mínimo un dígito.

    Args:
        password (str): Password que se desea validar.

    Returns:
        bool: True si el password cumple las tres condiciones, False en caso contrario.
    """
    assert isinstance(password, str), "Se esperaba un password del tipo str, pero se recibió: " +
         str(type(password))

    LONGITUD_MINIMA = 8
    SIMBOLOS = "#$%&!"
```



Hoja de respuesta adicional.
Por favor **escriba al inicio de la cuadrícula la pregunta que decide solucionar/continuar**, por ejemplo: "Continuo aquí la Pregunta 1."